

LECTURE 11

ENTERPRISE SYSTEM

ARCHITECTURE

DESIGN 2

LEK HSIANG HUI

LEARNING OBJECTIVES

At the end of this lecture, you should understand:

- Various components of enterprise systems security
- Software design patterns
- The activities of the implementation phase



ENTERPRISE SYSTEMS SECURITY

Enterprise
Systems
Security

Software
Design
Patterns

Implementation
Phase

DATABASE-LEVEL SECURITY

Password and sensitive information should be hashed instead of stored as clear text

- Commonly used hashing algorithm include MD5, SHA-1, SHA-2
- So that, the DB administrator would not be able to know the actual password

Idea:

- Instead of storing the original form, we apply a hashing algorithm to convert the password into a hashed form
- `MD5("password") = 5f4dcc3b5aa765d61d8327deb882cf99`
- When we need to verify whether the password supplied by user is correct, we apply the same hashing algorithm on the supplied password and compare the hash values

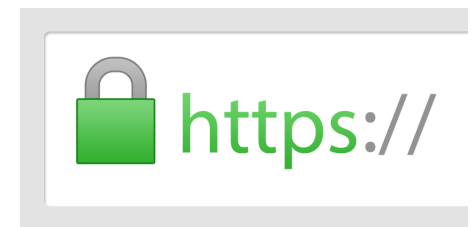


What do you think is one problem with the hashing approach we proposed in the previous slide?

TRANSPORT-LEVEL SECURITY

Hyper Text Transfer Protocol Secure (HTTPS)
should be used when a request involve sending of sensitive information (e.g. credit card details)

- This is to ensure that the data is encrypted before sending onto the network
- So the nodes in between will not be able to see the sensitive data



APPLICATION-LEVEL SECURITY

Of course, there should be security logic in order to provide proper authentication and authorization

- The framework can help to some basic mechanism e.g. only allowing logged in user to access a page
- However, we often require specific logic so the developers have to code out the logic
 - E.g. only the owner of a record is allowed to modify his/her record
 - Admin users can access all record



SOFTWARE DESIGN PATTERNS



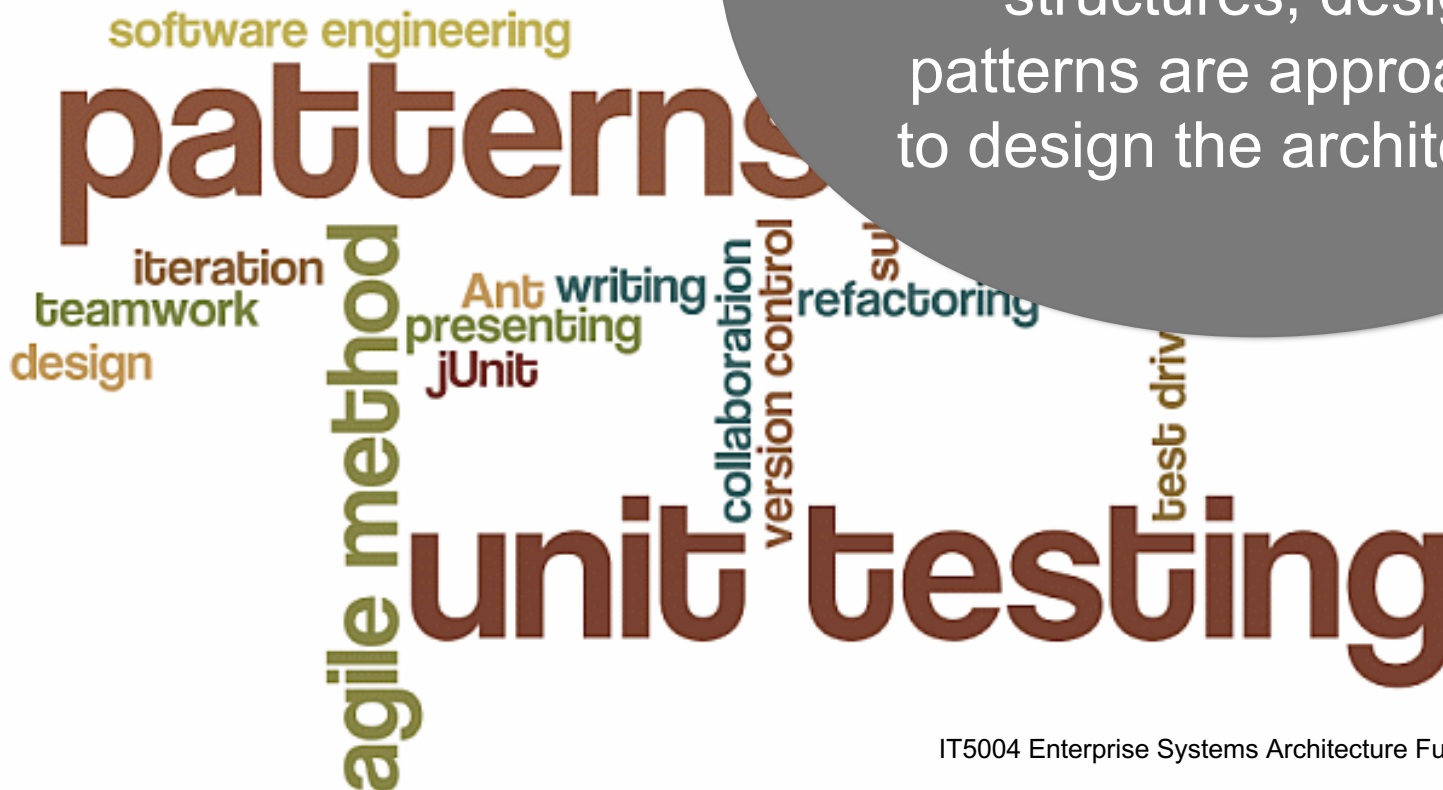
Enterprise
Systems
Security

Software
Design
Patterns

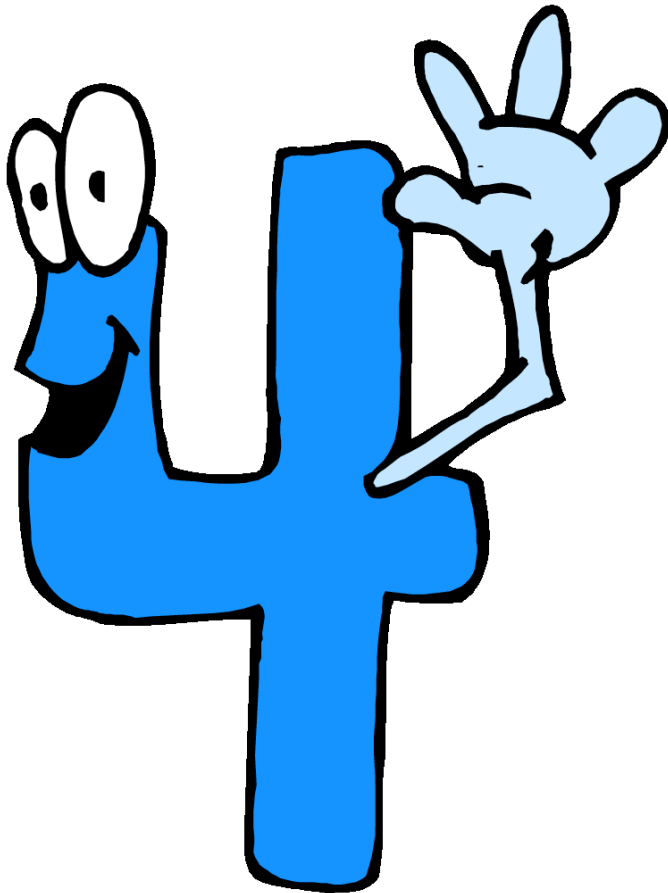
Implementation
Phase

SOFTWARE DESIGN PATTERNS

Often we find that many software systems have similar architectural structures, design patterns are approaches to design the architecture



ELEMENTS OF A PATTERN



4 elements of a pattern:

1. Name
2. Problem
3. Solution
4. Constraints

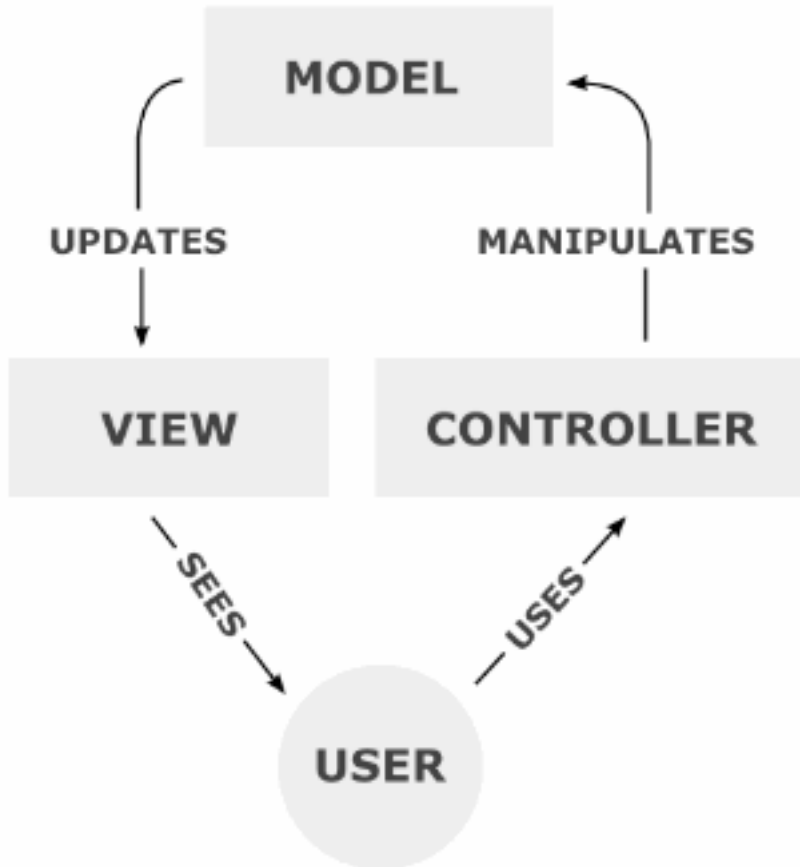
WHY STUDY DESIGN PATTERNS?



These design patterns are best practices which consider various factors. Some of them are natural solution to a problem

STRUCTURAL PATTERNS

MODEL/VIEW/CONTROLLER (MVC)



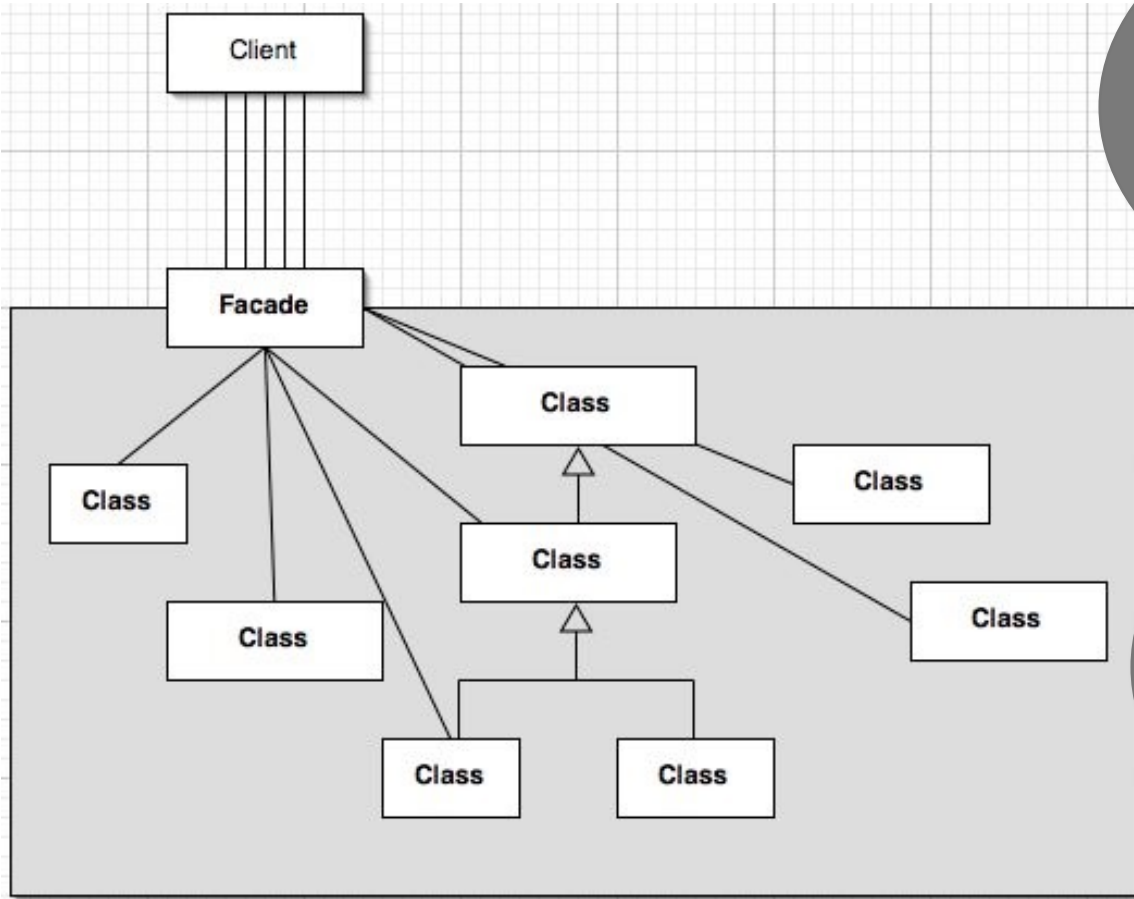
Model : Domain object (updates the view)

View : Presentation

Controller : receives commands from user and invoke the model's methods (direct traffic)

One of the most popular design pattern nowadays, commonly used for web applications

FACADE



Provides a unified interface to a set of interfaces in a subsystem

Idea is to wrap a complex subsystem with a simplified interface

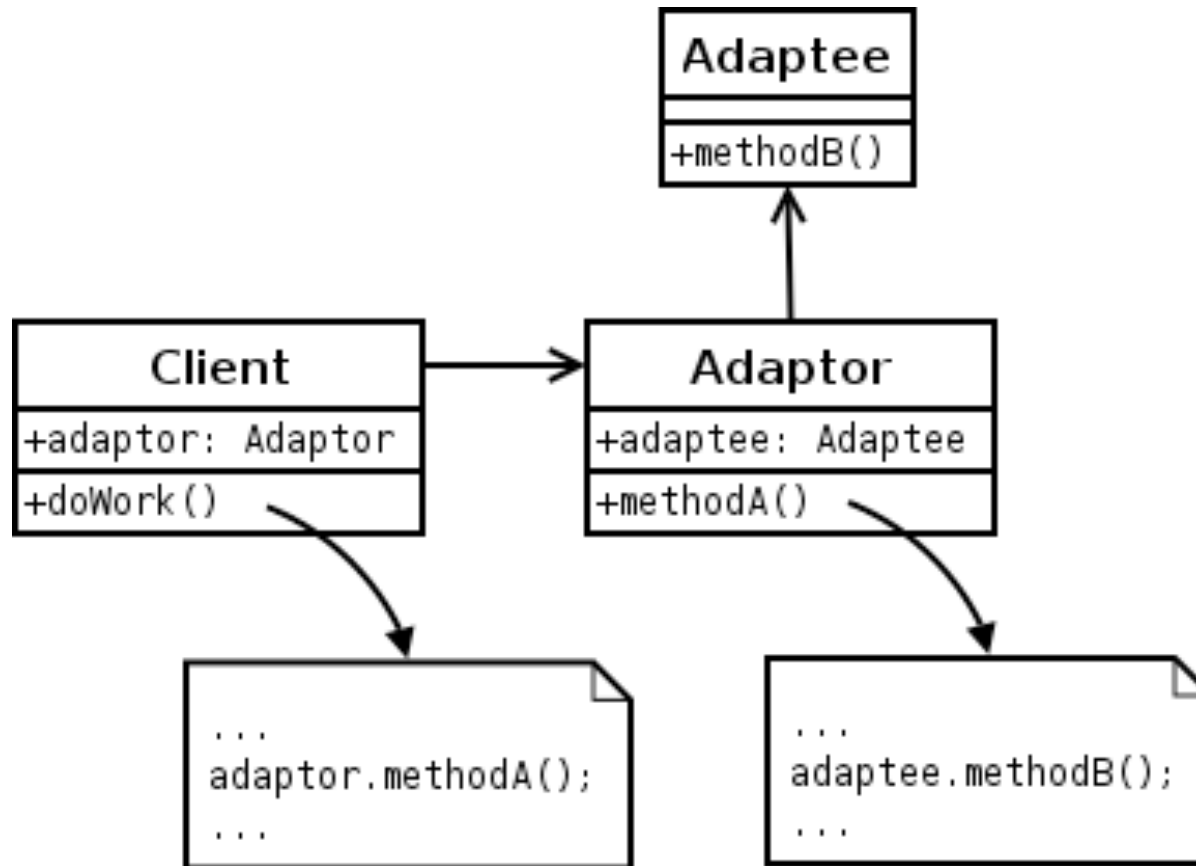
ADAPTOR (WRAPPER)

An adaptor allows classes to work together that normally could not due to incompatible interfaces

Idea is to convert the interface of a class into another interface that the client expects (Wrapper)



ADAPTOR (WRAPPER)



CREATIONAL PATTERNS

SINGLETON

Singleton
<u>- singleton : Singleton</u>
<u>- Singleton()</u>
<u>+ getInstance() : Singleton</u>

Restrict the
instantiation of a
class to one
object

Commonly used
for the Data
Access layer (e.g.
EntityManager in
JavaEE)

FACTORY

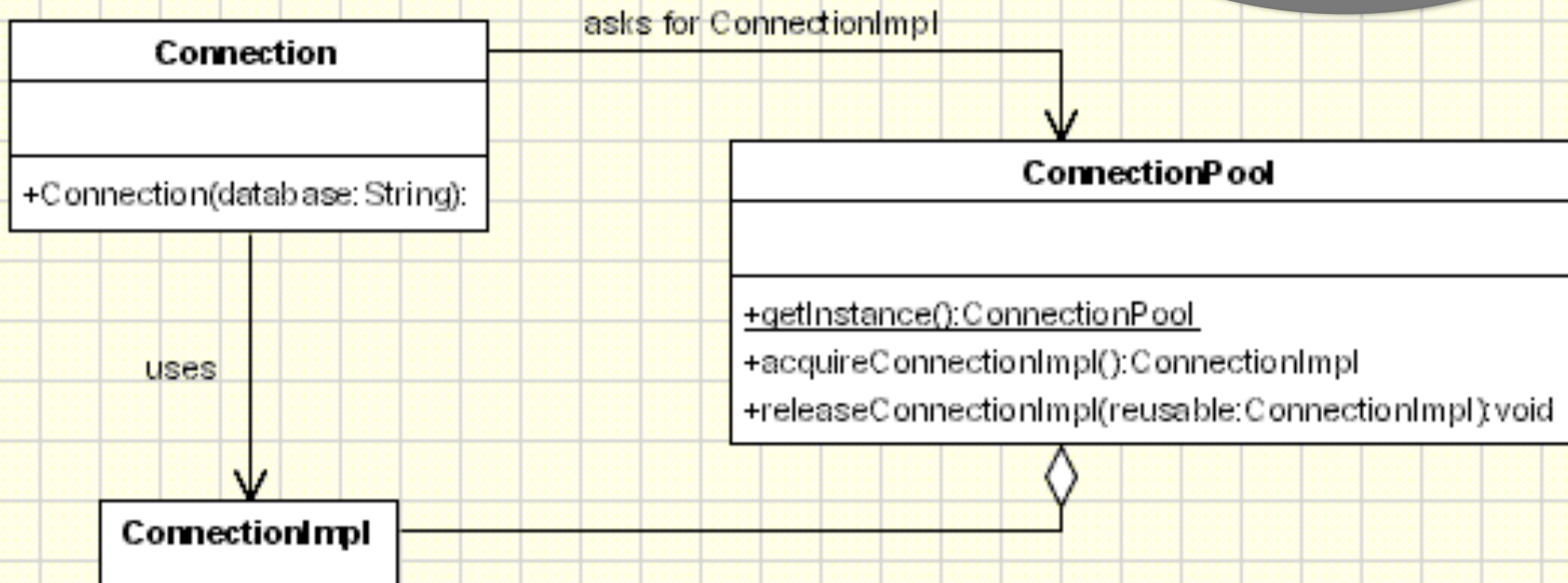
Make use of a
Factory to create
objects

```
class Connection:
    # static method
    def createConnection():
        # do some initialization
        # this process might requires tedious initialization
        # in which the factory will handle it
        return Connection()

# Use connection factory to create object rather than
# calling the constructor directly
connection1 = Connection.createConnection()
print(connection1)
```

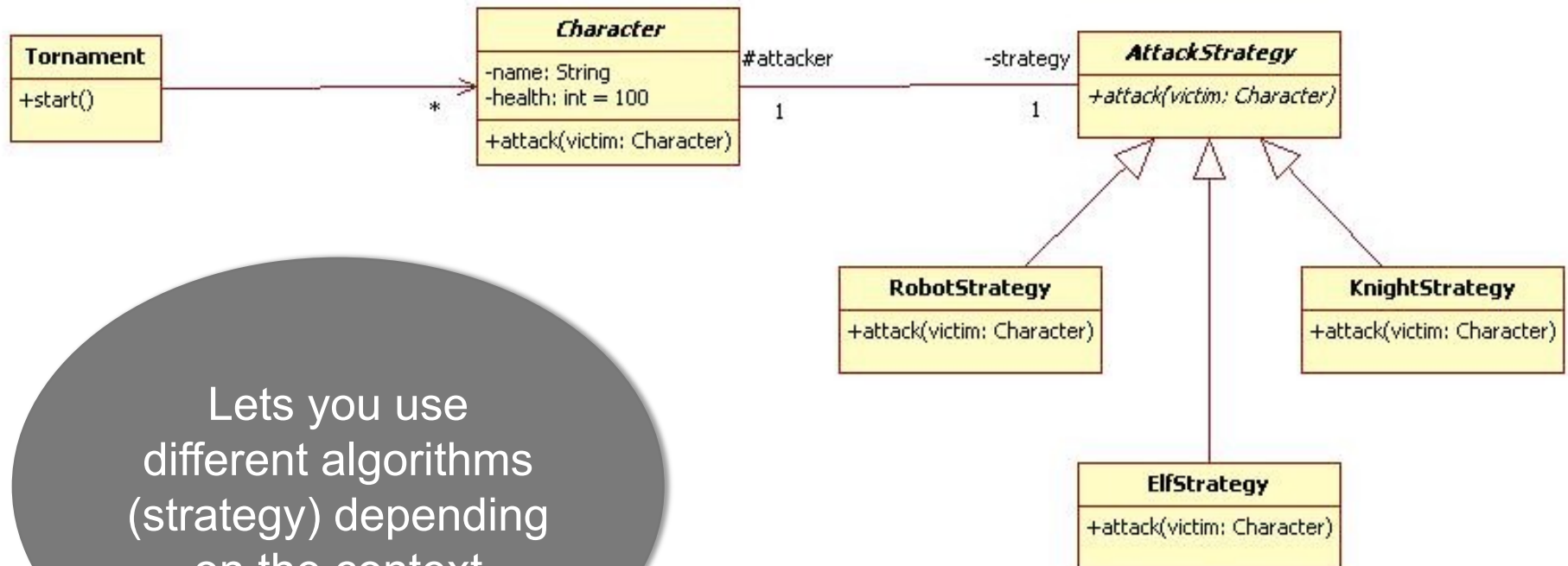
OBJECT POOL

Idea: want to reuse and share objects that are expensive to create



BEHAVIORAL PATTERNS

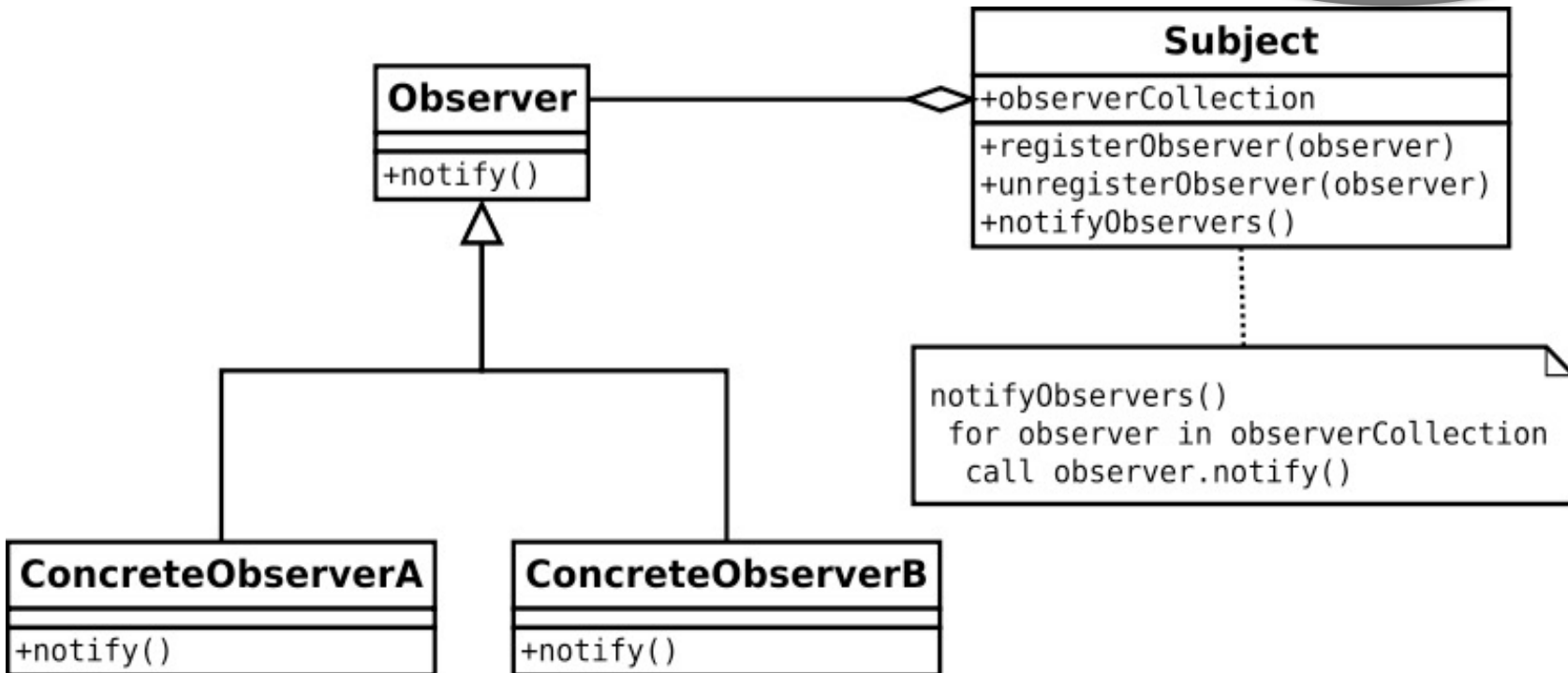
STRATEGY



Lets you use
different algorithms
(strategy) depending
on the context

OBSERVER

Allow objects to
notify other objects
about events
occurring



IMPLEMENTATION PHASE

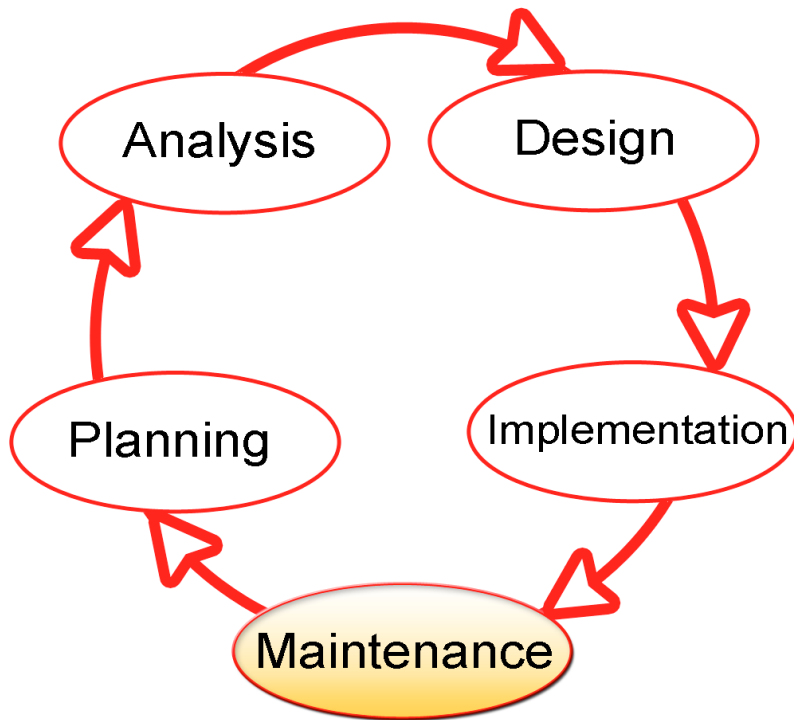


Enterprise
Systems
Security

Software
Design
Patterns

Implementation
Phase

IMPLEMENTATION PHASE



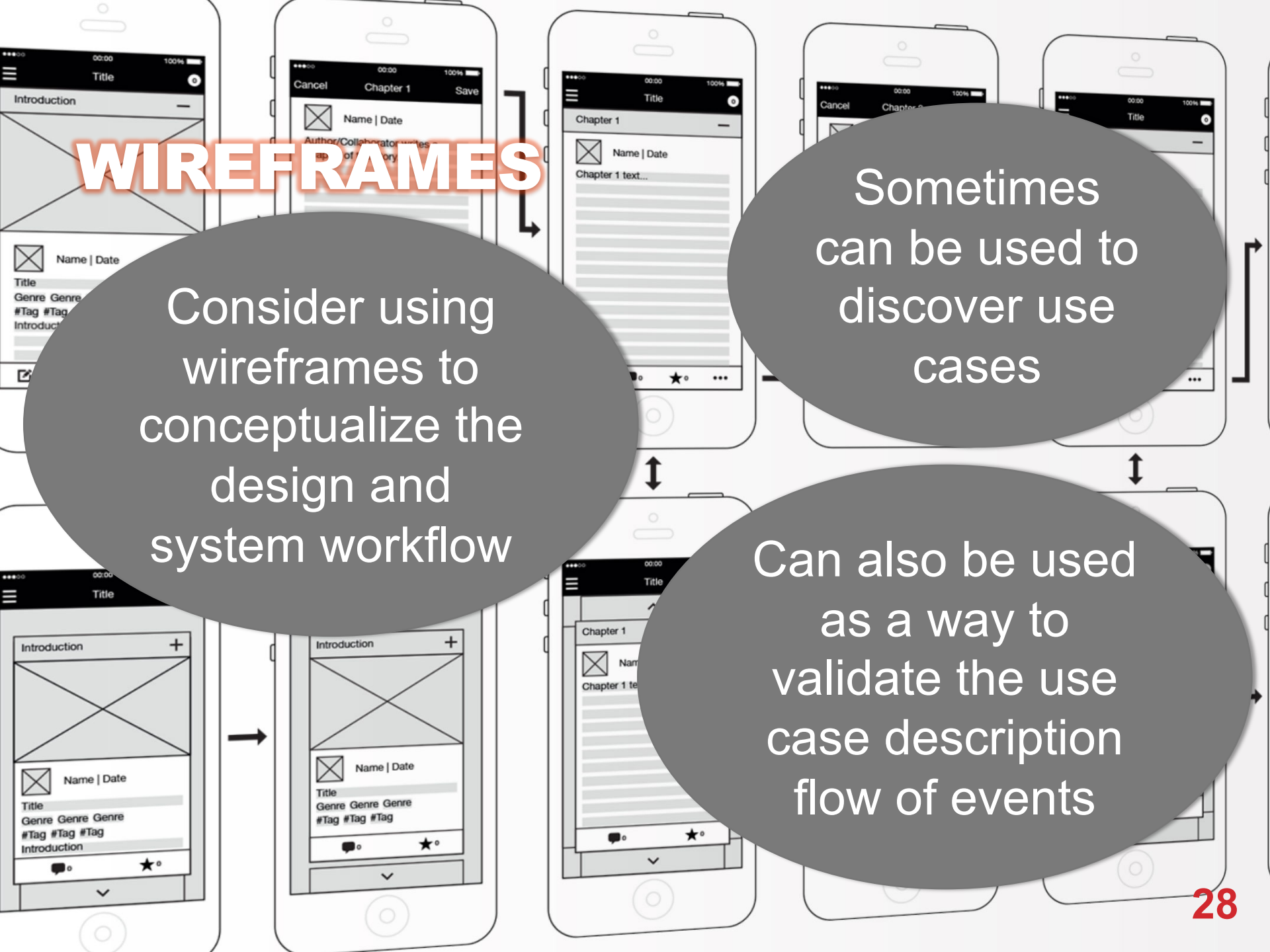
We can generalize SDLC into 5 different phases:

1. Planning
2. Analysis
3. Design
4. **Implementation**
5. Support/Maintenance

DESIGN AND CODING

Design (UI) and
Coding take place

WIREFRAMES



Consider using wireframes to conceptualize the design and system workflow

Sometimes can be used to discover use cases

Can also be used as a way to validate the use case description flow of events

MOCKUPS



Wireframes are typically filled with boxes and placeholders

The next step is to put in dummy photos and contents

And provide a product mockup which can be used to check with the client before developing the prototype

FRAMEWORKS AND COMMON STANDARDS

Recommended to use frameworks and common standards for development

- Ensure interoperability
- Ensure standardization and better integration with other system (or future systems)
- Speeds up development process



SUMMARY

Enterprise Systems Security

- Database security
- Transport-level security
- Application-level security

Software Design Pattern

- Structural patterns
- Creational patterns
- Behavioral patterns

SUMMARY

Implementation Phase Tips

- Wireframe, mockups, prototyping
- Use frameworks and common standards

WHAT'S NEXT?

Testing and Deployment